How to certify what a chess engine claims: WinTree inside Stockfish 19

Fabio Romano

November 3, 2026

Abstract

In a previous article I proposed a graph model of two-player games of perfect information and a backtracking algorithm, WinTree, which finds a winning strategy when one exists and, through a completeness theorem for draw strategies, a way to never lose. Here I report what happens when that algorithm is implemented and put inside a modern chess engine, Stockfish 19. Two optimizations suggested in the previous article turn out to be defective when implemented literally: the two-loop visiting order is unsound, and the "dejavu" memoization loses completeness at the root; I give counterexamples, the corrections, and a proof that the corrected dejavu rule (exact inheritance of the blocking vertices) preserves completeness at the root and the draw-strategy theorem. I then describe a generic C++ core with WinTree's outcome semantics, a chess adapter that makes certificates valid under the regulation rules (fifty-move rule, repetitions, tablebases), an independent verifier of proof objects, and the integration into Stockfish as a proof table consulted by alpha-beta and a prover invoked at the root. Measurements on 2000 mate positions, on 168 fortress positions and on more than 60 000 games show that the best-first traversal (df-pn) with WinTree's semantics certifies 96 % of the mates that Stockfish finds in ten seconds, that certified draws in practice are reductions to tablebase regions rather than fortresses, and that no playing-strength gain is measurable at any time control tried: the prover starts only after the engine's score is already decisive, and an earlier trigger finds proof trees that do not fit a per-move budget. The honest conclusion is that the result is a certification tool built over an unmodified Stockfish, not a stronger engine.

1 Introduction

The previous article [8] answered a theoretical question: whether a recursive, depth-first procedure can find a perfect strategy in a finite game graph with cycles, and whether its outcome can be exploited to find a strategy that never loses. The answer was positive, and the proofs were about the abstract procedure WinTreeRecursion: soundness (Theorem 3.1), completeness at the

root (Theorem 3.4), an upper bound on the depth to winning (Theorem 3.6) and completeness for draw strategies (Theorem 4.8). The article ended with two implementation ideas: visiting the already-solved (black) vertices first (Section 4.1 there), and remembering, for each unresolved vertex, the gray vertices that blocked its subtree, so that a later visit can be skipped when those vertices are still gray (the "dejavu" idea of page 21).

This article is the practical sequel. I wanted to know whether the algorithm, with its semantics of cycles as "unknown" and of proofs as explicit objects, could give something to a state-of-the-art chess engine: certified mates, certified draws in fortresses, and ultimately a stronger engine. The work was organized as a sequence of quantitative gates, each one deciding whether to continue, and the whole of it (code, test harnesses, raw results) is public [9]. What I found is the following.

1. Both optimizations of [8], implemented literally, are defective. The two-loop order of Section 4.1 is *unsound*: a child that becomes black during the recursion on a sibling is never read, and the procedure declares wins and non-wins that do not exist (Section 3.1). The dejavu memoization is *incomplete at the root*: a skipped vertex does not transmit its blockers to the vertex that skips it, and a vertex with an empty set of blockers is never explored again (Section 3.2). In the reference implementation the second defect also produces an unsound FALSE at the root. Both have small counterexamples and both were found by executing the code; the corrections are a few lines each, and random testing on 10^4 graphs shows no violation after them.

2. The corrected dejavu rule (the vertex that skips a child inherits the child's blockers, except itself) is proved to preserve completeness at the root, and Theorem 4.8 of [8] is proved for it (Section 4). The proof is not a rewording of the original one, which uses the instant at which all the vertices of a path are gray together; with a table of results that instant does not exist, and the argument goes through the expansions "derived from the root". A seemingly reasonable weaker rule (inherit only the deepest blocker) is shown not to be complete by a 12-vertex graph that random testing does not find.

3. As a traversal, the depth-first recursion of WinTree is inferior to a textbook best-first solver (df-pn, [7]) on chess mates. The production core therefore uses df-pn, but keeps WinTree's semantics: the four-valued outcome, the rule that a repetition is never a stored refutation, the validity intervals for the fifty-move rule and the verifier (Section 5). With the move-ordering information of Stockfish it certifies 96.1 % of the mates that Stockfish solves in ten seconds, against 64.1 % for the depth-first core.

4. Chess is not the abstract model. Section 6 describes what makes a certificate valid under the regulation rules: a state key without the fifty-move counter, validity intervals on that counter attached to every proven entry, tablebases as sinks, repetition handling, and an independent verifier that replays the proof objects on reconstructed positions, so that certainty never depends on the search or on hash keys.

5. Integrated into Stockfish 19 (Section 7) as a proof table consulted by alpha-beta and a prover invoked at the root, the system costs nothing measurable as a probe and gives nothing measurable as playing strength (Section 8): with the default trigger the prover starts only after the engine's score is decisive, and every game that reaches such a score is won by Stockfish alone; an earlier trigger starts the prover where it should, and the prover does not conclude within the budget. Certified draws beyond the coverage of the tablebases were not found.

Nothing above makes the previous article wrong: its theorems are about the procedure of Section 3 there, which is correct in every test, and the two defects are in the optimizations, whose proofs it did not give. What the present article adds is the price of those optimizations, their corrected form with proofs, and a measured account of what a proof-producing search can and cannot do next to a strong heuristic engine. I state the negative results with the same care as the positive ones, because they were the expensive part.

2 Preliminaries

I use the model and the notation of [8]. A *game graph* $G = \langle V, E, v_0, win, turn\rangle$ (Definition 2.6 there) is a directed graph whose sinks are the end positions, with $win : sinks(G) \to \{-1, 0, 1\}$ and $turn : V \to \{1, -1\}$ alternating along every arc; $G.Adj(v)$ is the set of successors of v. For a goal player $g \in \{1, -1\}$, $eval(v) = g$ means that g forces the win from v (Definition 2.7), $d_g(v) \in \mathbb{N} \cup \{\infty\}$ is the depth to winning (Definition 2.8), and $eval(v) = g \iff d_g(v) < \infty$ (Lemma 2.9). A vertex v is an *OR vertex* if $turn(v) = g$ and an *AND vertex* otherwise.

The procedure WinTreeRecursion(G, g, v) of Section 3 colors v GRAY, scans its successors, and ends by coloring v BLACK when $v.win$ is TRUE or FALSE, and WHITE when it is NIL; a gray successor yields NIL ("the winning tree can't have cycles"), an OR vertex stops at the first TRUE child, an AND vertex at the first FALSE child. I rely on the following results of [8]: $v.win$ = TRUE $\Rightarrow eval(v) = g$ (Theorem 3.1), $v.win$ = FALSE $\Rightarrow eval(v) \neq g$ (Corollary 3.3), $eval(v_0) = g \Rightarrow v_0.win$ = TRUE (Theorem 3.4), $v.win$ = TRUE $\Rightarrow v.d \geq d_g(v)$ (Theorem 3.6), the winning trees $WinTrees(G, g)$ of Definition 4.4 with their correctness and completeness (Lemmas 4.5 and 4.6: $T \in WinTrees(G, g) \Rightarrow eval(T.root) = g$, and $eval(v) = g \Rightarrow$ some winning tree is rooted at v), the strong completeness Lemma 4.7, and Theorem 4.8: if $\langle v_0, \ldots, v_n\rangle$ is a path whose vertices all have win = NIL after WinTreeRecursion(G, g, v_0), then $eval(v_n) \neq g$. Three facts about winning trees are used repeatedly: (F1) is Lemma 4.5, (F2) is Lemma 4.6, and (F3) every subtree of a winning tree is a winning tree, immediate from Definition 4.4.

In the implementations the three colors become *entries* of a table: PROVEN_WIN and PROVEN_NOWIN are the counterparts of a black vertex with win = TRUE and with win = FALSE, and are permanent; UNKNOWN_CYCLE is the counterpart of WHITE (unresolved only because of repetitions on the

current path) and is overwritten by every later expansion of the vertex; UNKNOWN_BUDGET marks a vertex whose exploration was cut by a limit on nodes, depth or time, and carries no information except the budget tried. A PROVEN_NOWIN entry means "g cannot force the win", which is not "draw": the other player may win.

3 Two defects of the optimized procedure

The reference implementation of [8] is a Java program of three files (`Engine`, `Position`, `Outcome`); it implements the two-loop procedure of Section 4.1 there, the dejavu idea of page 21 with a set of blocking vertices per vertex, and a root procedure `solve` which, when the opponent of g is to move at the root, treats a child left NIL as FALSE. The harness used throughout this section runs that program, a literal transcription of the procedure of Section 3 ("the transcription"), and a least-fixed-point solver that computes *eval* and d_g exactly ("the oracle") on the same graphs, and compares them vertex by vertex. Everything below was found by execution and is reproducible with one command [9].

3.1 P1: the two-loop order is unsound

The procedure of Section 4.1 of [8] scans the successors of v twice: a first loop reads the black ones, a second loop recurses on the others. In the second loop the whole update of $v.win$ and $v.d$ is guarded by "**if** $w.color \neq$ BLACK". Consider a child w that is not black during the first loop and *becomes black during the recursion on an earlier sibling* (because it is reachable from that sibling too: a transposition). It was not read in the first loop, and in the second loop it is skipped by the guard. Its outcome never enters the computation of v.

vertex	*turn*	successors	
v_0	1	v	root
v	-1	q_2, q_3	AND vertex (scanned in this order)
q_2	1	r, s_1	
r	-1	q_3	
q_3	1	s_2	
s_1			sink, $win(s_1) = 1$
s_2			sink, $win(s_2) = -1$

Table 1: Graph P1a, $g = 1$. The defender wins from v by playing $v \to q_3$, so $eval(v_0) = -1$.

Table 1 shows the smallest instance I found. Run WinTreeRecursion$(G, 1, v_0)$ with the two-loop procedure. At v (AND, initial value TRUE) no child is black, so the second loop recurses on q_2; inside q_2 the child r is explored, which explores q_3, which is refuted by s_2: q_3 becomes BLACK/FALSE, then r becomes BLACK/FALSE, then q_2 becomes BLACK/TRUE through s_1. Back

in v, the next child q_3 is black now, the guard skips it, and v keeps its initial value: $v.win$ = TRUE, hence $v_0.win$ = TRUE. But $eval(v_0) = -1$: Theorem 3.1 is violated. The mirror graph P1b (exchange the two players' roles: v is an OR vertex with initial value FALSE, q_3 becomes BLACK/TRUE during the recursion on q_2 and is then skipped) leaves $v.win$ = FALSE with $eval(v) = 1$, violating Corollary 3.3. The transcription of Section 3 is correct on both graphs. The defect is in the pseudocode of Section 4.1 (lines 37–55 there), not only in the Java.

In chess the configuration requires two paths from v to the same child q_3, of lengths one and at least three plies; since every move changes the position, the attacker must undo his own move, so the long path has five plies (for example a defending king going e8–f8–f7–e7 against e8–e7 while the attacking king goes e1–f1–e1, with equal castling rights on both paths). Transpositions of this kind are common in endgames.

The correction is simple: the second loop must skip the *recursion* on black children but always *read* their outcome, and, not to count twice the children read in the first loop, it records them in a set D and iterates on $G.Adj(v) \setminus D$ (Section 3.3, lines 16 and 32).

3.2 P2: the dejavu memoization is incomplete at the root

The dejavu idea is implemented as follows in the reference program. Every vertex v visited by the recursion gets a set $v.dejavu$, initially empty. When a gray successor q of the current vertex v is met, q is added to $x.dejavu$ for every vertex x on the current path from v down to q, q excluded: those are the vertices whose subtree contains the back arc to q, and q is a *blocker* for each of them. When a white successor w is met, w is explored again only if some element of $w.dejavu$ is no longer on the path; if all its blockers are still gray, the earlier outcome NIL cannot have changed and w is skipped.

Two things are missing. (a) When w is skipped, the blockers of w are not added to the sets of v and of its ancestors: the subtree of v now depends on those gray vertices too, and the sets do not say so. (b) A white vertex with an empty set is never explored again. Table 2 gives the smallest instance I found.

Run the procedure from v_0. Through b, a_2, r and r_2 it reaches q, whose only successor a_2 is gray: q ends WHITE with $q.dejavu = \{a_2\}$, and a_2 is added to the sets of r_2 and r as well. Back in r, the next child p is explored; its only successor q is white and its blocker a_2 is still gray, so q is skipped, and p ends WHITE with $p.dejavu = \emptyset$: this is (a). Then a_2 becomes BLACK/TRUE through y and b becomes BLACK/FALSE through z. At the root, the second child p_0 leads to p, which is white with an empty set: it is skipped, by (b), and v_0 ends NIL, although $eval(v_0) = 1$. Theorem 3.4 is violated. The reference program then makes it worse at the root (I call this P2b): in `solve`, when the opponent is to move at the root, a child left NIL is treated as FALSE, which is justified by Theorem 3.4 and is wrong as soon as Theorem 3.4 fails; random testing produced a root declared FALSE with $eval(v_0) = g$ (seed 191 of the harness). The practical consequence is the loss of the guarantee of Theorem 4.8:

vertex	*turn*	successors	
v_0	1	b, p_0	root
b	-1	a_2, z	
a_2	1	r, y	
r	-1	r_2, p	
r_2	1	q	
q	-1	a_2	
p	1	q	
p_0	-1	p	
y			sink, $win(y) = 1$
z			sink, $win(z) = -1$

Table 2: Graph P2, $g = 1$. $eval(v_0) = 1$: the attacker plays $v_0 \to p_0$, the defender is forced to p, and $p \to q \to a_2 \to y$ wins.

the move chosen as "not losing" through a NIL child can be a lost move.

The correction treats a skip as a *virtual hit* on each blocker: when w is skipped, every $y \in w.dejavu$ is added to $x.dejavu$ for every vertex x on the path from v down to y, y excluded, exactly as if the subtree of v had met y gray (Section 3.3, lines 25–30). After it, a white vertex has an empty set only if its subtree, explored with its own ancestors as the only blockers, contains no winning tree at all, and then skipping it forever is right. Section 4 proves this.

3.3 The corrected procedure

The following is the procedure of Section 4.1 of [8] with both corrections; it is what the published Java implements. NIL denotes an attribute not yet initialized, as there; a vertex whose color is NIL has never been visited. The set *path* is the set of gray vertices, ordered from the current vertex down to the root.

```
 1:  procedure WinTreeRecursion(G, g, v)
 2:    v.color ← GRAY;    v.dejavu ← ∅
 3:    if v ∈ sinks(G) then                                  ▷ base of recursion
 4:      if win(v) = g then v.win ← TRUE; v.d ← 0 else v.win ← FALSE;
v.d ← ∞ end if
 5:    else                                                  ▷ recursive step
 6:      if turn(v) = g then v.win ← FALSE; v.d ← ∞ else v.win ← TRUE;
v.d ← 0 end if
 7:      end ← FALSE;    D ← ∅                  ▷ D: children read in the first loop
 8:      for each w ∈ G.Adj(v), in order, while not end do
 9:        if w.color = BLACK then
10:          D ← D ∪ {w}
11:          if w.win ≠ (turn(v) ≠ g) then v.win ← w.win; v.d ← w.d + 1;
end ← TRUE
12:          else if turn(v) ≠ g and w.d + 1 > v.d then v.d ← w.d + 1
13:          end if
14:        end if
```

```
15:       end for
16:       for each w ∈ G.Adj(v) \ D, in order, while not end do          ▷ P1
17:         if w.color = GRAY then
18:           win ← NIL; d ← ∞
19:           for each x ∈ path from v down to w, w excluded do
20:             x.dejavu ← x.dejavu ∪ {w}
21:           end for
22:         else
23:           if w.color = NIL or (w.color = WHITE and w.dejavu ⊈ path) then
24:             WinTreeRecursion(G, g, w)
25:           else if w.color = WHITE then                    ▷ P2: w skipped
26:             for each y ∈ w.dejavu do
27:               for each x ∈ path from v down to y, y excluded do
28:                 x.dejavu ← x.dejavu ∪ {y}
29:               end for
30:             end for
31:           end if
32:           win ← w.win; d ← w.d + 1                  ▷ w.win is NIL if w is white
33:         end if
34:         if win ≠ (turn(v) ≠ g) then
35:           v.win ← win; v.d ← d; if v.win ≠ NIL then end ← TRUE end if
36:         else if turn(v) ≠ g and d > v.d then v.d ← d
37:         end if
38:       end for
39:     end if
40:     if v.win = NIL then v.color ← WHITE else v.color ← BLACK end if
41:   end procedure
```

Line 16 is the P1 correction (the loop runs on $G.Adj(v) \setminus D$, that is, on every child not read in the first loop whatever its color now, and reads black children too, through line 32); lines 25–30 are the P2 correction, the virtual hit on each blocker of the skipped w. The comparison "$win \neq (turn(v) \neq g)$" of line 34 is the one of the original procedure: an OR vertex takes a TRUE or a NIL, an AND vertex a FALSE or a NIL. Everything else, including the computation of $v.d$, is unchanged.

3.4 Random testing

The harness generates 10^4 random game graphs with a seed ($n \in [10, 40]$ vertices, out-degree in $[1, 3]$, a density of back arcs in $[0, 0.7)$ and a fraction of sinks in $[0.1, 0.4)$; arcs join vertices of opposite turn, so there are no self-loops) and runs each implementation for both goals: 20 000 searches, of which 4142 have $eval(v_0) = g$. For every search it checks soundness (every TRUE vertex has $eval = g$ and every FALSE vertex has $eval \neq g$, by the oracle), completeness at the root (Theorem 3.4), and Theorem 4.8 with a model checker: from the root, the defender chooses NIL children of NIL vertices or FALSE children, the attacker chooses anything, and the check fails if a choice is missing or a sink won by g is reached. Table 3 counts the searches with at least one violation.

implementation	root not TRUE	unsound TRUE	unsound FALSE	root incomplete (Thm. 3.4)	draw strategy (Thm. 4.8)
original Java	15 942	7	45	89	129
P1 corrected	15 859	0	1	1	2
P1 + P2 corrected	15 858	0	0	0	0
Section 3 transcription	15 858	0	0	0	0

Table 3: Random testing of the Java implementations on 10^4 graphs $\times$ 2 goals. Each cell counts the searches with at least one violation of the stated property.

Three readings. The transcription never violates anything, so the oracle, the transcription and the model checker are consistent with the theorems. P1 appears spontaneously in random graphs (7 unsound TRUE and 45 unsound FALSE), and the correction removes all of them except one, which is P2b (seed 191: the root is declared FALSE because a child left NIL by a skip has $eval = g$); with the dejavu disabled the P1-corrected version has no violation at all, which isolates the cause. With both corrections the Java has the same root outcomes as the transcription in every search, and a differential comparison with the C++ core of Section 5 on 20 000 graphs (the same 10^4 and 10^4 acyclic ones) shows identical states, depths and visiting orders on the acyclic graphs and no disagreement on proven vertices on the cyclic ones.

Remark 3.1. A third defect, not reproduced by the harness, is a latent null-pointer dereference in the move-selection routine of the reference program (`bestMove`), which compares the opponent's evaluation of a child without checking that the child was evaluated. It is outside the scope of the theorems and is only recorded here.

4 Dejavu with exact inheritance of blockers

This section proves that the corrected dejavu rule preserves completeness at the root and Theorem 4.8. The proofs are stated for the search with a proof table of the C++ core, because the counterexample for the weaker rule and the measurements were made there; Remark 4.13 explains why they apply verbatim to the Java procedure of Section 3.3.

4.1 The search with a table

The search explores G from v_0 depth-first as WinTreeRecursion does. The current path is $path[0..k-1]$ with $path[0] = v_0$; its vertices are the gray vertices of [8]. For a set of plies $B \subseteq \{0, \dots, k-1\}$ write $path[B] = \{path[q] : q \in B\}$; since the path is simple, $q \mapsto path[q]$ is injective. The frame at ply k keeps a set $B_k \subseteq \{0, \dots, k-1\}$ of plies, the *blockers* met in its subtree. A vertex w reached at ply k is handled by $visit(k)$:

(V1) if $w = path[q]$ for some $q < k$ (a gray vertex): outcome NIL, and $B_{k-1} \leftarrow B_{k-1} \cup \{q\}$;

(V2) if the table has a PROVEN_WIN or PROVEN_NOWIN entry for w: outcome TRUE or FALSE, no recursion;

(V3) if the table has an UNKNOWN_CYCLE entry for w whose blockers are all on the current path: outcome NIL (the *skip*), and the parent inherits by the *inheritance rule* defined below;

(V4) otherwise w is *expanded* by $rec(k)$.

$rec(k)$ for a vertex u: (R1) if u is a sink, write a PROVEN_WIN entry if $win(u) = g$ and a PROVEN_NOWIN entry otherwise; (R2) $path[k] \leftarrow u$, $B_k \leftarrow \emptyset$; (R3) scan the children, first those with a PROVEN_WIN/PROVEN_NOWIN entry (read without recursion), then the others in order, each through $visit(k+1)$: an OR vertex becomes TRUE and stops at the first TRUE child, an AND vertex becomes FALSE and stops at the first FALSE child, and a NIL child sets a flag *nil* (in the mode that looks only for wins, an AND vertex also stops at the first NIL child); when an expanded child ends NIL, its frame has already executed $B_k \leftarrow B_k \cup (B_{k+1} \setminus \{k\})$; (R4) at the end, TRUE/FALSE produce a PROVEN_WIN/PROVEN_NOWIN entry; otherwise u ends NIL and gets an UNKNOWN_CYCLE entry carrying its blockers, and if $k > 0$ then $B_{k-1} \leftarrow B_{k-1} \cup (B_k \setminus \{k-1\})$. One verifies by induction that $B_k \subseteq \{0, \ldots, k-1\}$, so $path[B_k]$ is a set of proper ancestors of u, all gray.

In the C++ core an UNKNOWN_CYCLE entry stores $p = \max B$ and a hash of the prefix $path[0..p]$ at the time of writing; the skip condition of (V3) is "$p < k$ and the current prefix $path[0..p]$ has the same hash", which, up to collisions, says that the blocking vertices of the entry are the same vertices, at the same plies, all gray now. The exact set B is kept beside the entry (*variant A*). An *inheritance rule* says what the parent $path[k-1]$ of a skipped child inherits; let B' be the set of the skipped entry. The rule is *admissible* if

$$B_{k-1} \supseteq B' \setminus \{k-1\}. \tag{H}$$

Variant A inherits exactly $B' \setminus \{k-1\}$; *variant B* inherits the whole prefix $\{0, \ldots, \min(p, k-2)\}$, which contains $B' \setminus \{k-1\}$ because $B' \subseteq \{0, \ldots, p\}$. The rule first implemented in the core inherited only $\{p\}$ (or $\{k-2\}$ when $p = k-1$); it is not admissible, and Section 4.4 shows that it is not complete.

4.2 Invariant and conservation

Definition 4.1 (Blockers of a NIL outcome)**.** For an UNKNOWN_CYCLE entry of w written at the end of an expansion with path P and set B, the blockers are $blocc := P[B]$; for a frame at ply k ending NIL with set B_k, $blocc := path[B_k]$.

Invariant (I). For every UNKNOWN_CYCLE entry of w, and for every frame of w ending NIL: every $T \in WinTrees(G, g)$ with $T.root = w$ has $nodes(T) \cap blocc \neq \emptyset$, and $blocc \subseteq Reach(w)$, the set of vertices reachable from w. In particular $B = \emptyset$ means that no winning tree is rooted at w, hence $eval(w) \neq g$ by (F2).

(I) is a static property of the graph: once true it stays true, whatever happens to the table later (vertices becoming proven, entries overwritten, later searches).

Lemma 4.2 (Soundness)**.** *A* PROVEN_WIN *entry of v implies $eval(v) = g$; a* PROVEN_NOWIN *entry of v implies $eval(v) \neq g$.*

Proof. A proven entry is written only in (R1) for a sink, or in (R4) from TRUE/FALSE outcomes of the children, which are proven entries read in (V2) or outcomes of recursive expansions; gray vertices and skips produce only NIL. The claim follows by induction on the order of the writes, with the argument of Theorem 3.1 and Corollary 3.3 of [8]. □

Lemma 4.3 (Conservation of (I))**.** *With an admissible inheritance rule, (I) holds for every* UNKNOWN_CYCLE *entry written and for every frame ending* NIL *(up to collisions of the prefix hash).*

Proof. By induction on the execution: assume (I) for every entry written before and for every call completed before the end of $rec(k)$ for u, and prove (I) for the frame of u; the entry written in (R4) copies B_k and the property is static, so it holds for the entry forever. Let u end NIL at the end of (R3). Suppose, for a contradiction, that some winning tree rooted at u avoids $path[B_k]$, and let T be one of minimal height.

Step 1 (u does not repeat). $u \notin nodes(T) \setminus \{T.root\}$: otherwise the subtree rooted at that occurrence would be a winning tree (F3), would avoid $path[B_k]$, and would be lower.

Step 2 (a NIL *child with a subtree in T).* There is a child w of u, examined in (R3) with outcome NIL, such that T has a subtree T_w with $T_w.root = w$. If u is an OR vertex, u NIL means that no child gave TRUE, so the scan was not interrupted and all children were examined; T has exactly one child subtree, rooted at some $w \in G.Adj(u)$ (Definition 4.4); the outcome of w is not TRUE (none is) and not FALSE (Lemma 4.2 and (F1): no winning tree is rooted at a vertex with $eval \neq g$, but T_w is one); hence NIL. If u is an AND vertex, u NIL means that an examined child w gave NIL (that is what sets the flag), and by Definition 4.4 T has a subtree rooted at every successor, in particular at w.

Step 3 (contradiction). T_w is a winning tree (F3) rooted at w which avoids $path[B_k] \cup \{u\}$ (Step 1, and T avoids $path[B_k]$). Three cases, by the origin of the NIL of w. (V1): $w = path[q]$ with $q \in B_k$, so $w \in path[B_k] \cap nodes(T)$. Contradiction. (V4): by the inductive hypothesis every winning tree rooted at w meets $path[B_{k+1}]$ (the path of w is $path[0..k]$, the current one); by (R3), $path[B_{k+1}] \subseteq path[B_k] \cup \{path[k]\} = path[B_k] \cup \{u\}$, so T_w meets $path[B_k] \cup \{u\}$. Contradiction. (V3): the entry was written with a path P and a set B'; by the inductive hypothesis every winning tree rooted at w meets $P[B']$; the skip condition gives $P[B'] = path[B']$ (the same vertices), and by (H) $path[B'] \subseteq path[B_k] \cup \{u\}$, so T_w meets $path[B_k] \cup \{u\}$. Contradiction.

Hence no winning tree rooted at u avoids $path[B_k]$. Finally $path[B_k] \subseteq Reach(u)$: in (V1) $w = path[q] \in G.Adj(u)$; in (V3) and (V4) the blockers of w are reachable from w by the inductive hypothesis, and $w \in G.Adj(u)$. □

The rule "inherit only the deepest blocker" leaves the proof in case (V3): $B' \setminus \{k-1\}$ may contain a ply $q < p$, and $path[q]$ is a blocker of w that the parent forgets. As long as q stays below $\max B_k$ the loss is hidden by the prefix condition, which covers all of $path[0..\max B_k]$; but when the propagation of (R4) reaches the ancestor at ply p, that ancestor removes its own ply and keeps nothing of w: its entry, and those above it, get a p that is too small, down to -1.

4.3 Completeness at the root

Theorem 4.4 (Completeness at the root)**.** *With an admissible inheritance rule, if $eval(v_0) = g$ then the search without budget ends with a* PROVEN_WIN *entry for v_0.*

Proof. The root is at ply 0 and $B_0 \subseteq \emptyset$. If its outcome were NIL, by Lemma 4.3 every winning tree rooted at v_0 would meet $path[\emptyset] = \emptyset$: there would be no winning tree rooted at v_0, against (F2). If it were FALSE, $eval(v_0) \neq g$ by Lemma 4.2. Without budget there are no cuts. It remains TRUE. □

Corollary 4.5 (Strong version, analogue of Lemma 4.7)**.** *Let v be the vertex examined by a call visit(k) that does not fall in case (V1), and let $S = path[0..k-1] \cap Reach(v)$ be the set of gray vertices reachable from v at that moment. If there exists $T \in WinTrees(G, g)$ with $T.root = v$ and $nodes(T) \cap S = \emptyset$, the call returns* TRUE*.*

Proof. The outcome is not FALSE (Lemma 4.2 and (F1)). If it were NIL: in (V3) every winning tree rooted at v meets $P[B'] = path[B'] \subseteq S$ (static invariant and skip condition); in (V4) every winning tree rooted at v meets $path[B_k] \subseteq S$ (Lemma 4.3, with $path[B_k] \subseteq Reach(v)$). In both cases no winning tree rooted at v avoids S, against the hypothesis. It remains TRUE, read in (V2) or produced by (V4). □

Remark 4.6 (What is not preserved)*.* A skipped vertex w may be provable *at that moment*: for example a child c of w may have become PROVEN_WIN after w was stored, with a proof tree passing through the gray blocker y (consistent with (I): that tree meets y). The skip leaves w NIL and loses *local* completeness, the same phenomenon as the white vertices with $eval = g$ of [8]. Completeness at the root does not suffer: (I) for the parent stays true, and a minimal winning tree from the root cannot pass twice through y. The correctness of the skip does not depend on "w is unsolvable now" but only on "the blockers of w are on the path". A false positive of the prefix hash makes the search skip w with a different prefix: $P[B'] \neq path[B']$ in general, Step 3 does not apply, and completeness at the root may fail; soundness cannot, because a skip produces only NIL. The hash has 32 bits in the core (with 16 bits false positives already appeared in 10^4 graphs).

id	type ($g = 1$)	succ.	d_g	role
0	OR	1, 5	7	root r; wins only through 5
1	AND	2, 3	∞	m: leads the search to b, then becomes FALSE through 3
2	OR	5, 4	1	b: the "high" blocker of w; explores x, then becomes TRUE through 4
3	sink, $win = -1$		∞	f: refutes m and u
4	sink, $win = 1$		0	s: the win of b
5	AND	6	6	n: the way to x from b and from the root
6	OR	7, 8	5	x: the "deep" blocker of w; ends with $p = -1$
7	AND	9, 3	∞	u: under it w is expanded; u becomes FALSE and propagates nothing
8	AND	9	4	u_2: under it w is skipped
9	OR	10, 11	3	w: NIL with two blockers, x (ply 4) and b (ply 2)
10	AND	6	6	c_1: parity bridge to x
11	AND	2	2	c_2: parity bridge to b

Table 4: Graph P4, $g = 1$ (the turn of a vertex is 1 for OR and -1 for AND). The only winning tree from the root is $0 \to 5 \to 6 \to 8 \to 9 \to 11 \to 2 \to 4$ and passes through vertex 2, the blocker that the deepest-blocker rule forgets.

4.4 The deepest-blocker rule is not complete

Proposition 4.7. *With the rule that inherits only the deepest blocker, the search on the graph of Table 4 ends with the root* UNKNOWN_CYCLE*, although* $eval(0) = 1$ *and* $d_1(0) = 7$*. With variant A or B it ends with* PROVEN_WIN *and* $d = 7$*.*

I only sketch the mechanism; the full trace is in the repository. Under u (vertex 7) the vertex w is expanded with blockers x and b; u is then refuted through f and, being FALSE, propagates nothing. Under u_2 (vertex 8), reached with the same prefix, w is skipped and u_2 inherits only the deepest blocker x; when the propagation reaches x, it removes its own ply and has nothing left of w: the entry of x gets $p = -1$, which means "no winning tree from x" and is false ($d_1(6) = 5$). An entry with $p = -1$ satisfies the skip condition at every ply, so x is skipped forever, also from the root through n, and the root ends unresolved. Random testing with 10^4 graphs did not find this: the defect needs a vertex with two blockers at different plies, skipped under a sibling that is refuted. The graph was built by hand from the failed proof, and a second variant (P4bis, $d_1(0) = 10$) checks the other branch of the rule; both are regression tests of the core. On the 20 000 random searches the total number of expansions is 135 087 with the original rule, 134 562 with variant A, 142 439 with variant B and 217 370 without dejavu: the exact rule costs slightly less than the defective one, because the latter overestimates when the deepest blocker is the parent itself.

4.5 Theorem 4.8 for variant A

Theorem 4.8 of [8] is what makes the search usable for not losing: if the search from v_0 leaves a path of NIL vertices, the last of them is not a forced win for g, and the defender who follows NIL children never loses. Its proof uses the instant at which all the vertices of the path are gray together. With a table that instant does not exist: a vertex may be expanded several times, each expansion overwrites its UNKNOWN_CYCLE entry, and a skip reuses an entry written along a different path. The following proof goes through the expansions "derived from the root".

Hypotheses: the search runs in the mode that explores all children of an AND vertex (the mode used for draw strategies), without budget, on an empty table, without the fifty-move rule, without hash collisions and without replacement of entries. After the search, every visited vertex has a PROVEN_WIN entry (set W), a PROVEN_NOWIN entry (set F) or an UNKNOWN_CYCLE entry (set N); vertices never expanded have no entry; proven entries are permanent and an UNKNOWN_CYCLE entry is overwritten by every later expansion of its vertex, so $v \in N$ if and only if all its expansions end NIL. Let R be the set of vertices reachable from v_0 along paths of vertices of N ($R = \emptyset$ if $v_0 \notin N$).

Theorem 4.8 (Completeness for draw strategies with a table)**.** *After a complete search with $v_0 \in N$, $eval(v) \neq g$ for every $v \in R$; equivalently, for every path $\langle v_0, \ldots, v_n \rangle$ of G with all $v_i \in N$, $eval(v_n) \neq g$.*

This is the reading made by the verifier of Section 6.4 and by the model checker of the tests: the defender chooses children in N from a vertex in N, or children in F; the attacker chooses anything; the check fails if a choice is missing or a sink won by g is reached. If the theorem holds, from an attacker's vertex in N no child is in W (it would have $eval = g$) and no child is a sink won by g; from a defender's vertex in N there is a child with $eval \neq g$, which is not in W by soundness and has an entry (a NIL vertex in this mode examines all its children), so it is in N or in F; vertices in F have children in F by the rule of the procedure (all of them at an OR vertex, at least one at an AND vertex), and a sink in F is not won by g. Conversely, if the model checker passes, the defender has a strategy that stays in $R \cup F$ or ends in sinks not won by g, so g forces the win from no vertex of R.

Definition 4.9 (Expansions, modes, derived expansions)**.** An *expansion* E of a vertex x is a call *rec* on x. If E ends NIL, its *support* $supp(E)$ is the set of its blockers as vertices (a subset of the gray ancestors at the time of E), and every child c of x examined in E with outcome NIL has one of three *modes*: GRAY (c is on the path), EXPAND (c is expanded inside E by a call ending NIL) or SKIP (dejavu skip of the current UNKNOWN_CYCLE entry of c, written by the last expansion of c before the examination). In the modes EXPAND and SKIP the *used expansion* E^c is, respectively, the nested call and the expansion that wrote the skipped entry; in both cases E^c ends NIL. Π_0 is the smallest set of expansions such that (i) the expansion E_0 of the root (which ends NIL

because $v_0 \in N$) is in Π_0, and (ii) if $E \in \Pi_0$ and a child c is examined in E in the mode EXPAND or SKIP, then $E^c \in \Pi_0$. Every $E \in \Pi_0$ has a *derivation chain* $E_0 = E^0, E^1, \ldots, E^s = E$, where E^{j+1} is the expansion used by the examination, in E^j, of the child x_{j+1} of the vertex x_j of E^j ($x_0 = v_0$); the *chain vertices* $x_0, \ldots, x_s$ are joined by the arcs $x_j \to x_{j+1}$ of G. All expansions of Π_0 end NIL. R_0 is the set of vertices having at least one expansion in Π_0.

Lemma 4.10 (Blockers are chain vertices)**.** *Let $E \in \Pi_0$ with chain $E^0, \ldots, E^s = E$. Then $supp(E) \subseteq \{x_0, \ldots, x_{s-1}\}$; in particular $supp(E) \subseteq R_0$.*

Proof. Let $c \in supp(E)$. By descending induction on j from s to 0: if $c \in supp(E^{j+1})$ and $c \neq x_j$, then $c \in supp(E^j)$. Indeed, if the mode of x_{j+1} in E^j was EXPAND, at the end of E^{j+1} the parent inherits by (R4) $B_{x_j} \leftarrow B_{x_j} \cup (B_{x_{j+1}} \setminus \{\text{ply of } x_j\})$, and the plies refer to the same path. If it was SKIP, the inheritance rule adds to B_{x_j} the set stored with the entry written by E^{j+1} (variant A: exactly $B_{x_{j+1}}$; variant B: a superset) minus the ply of x_j, and the skip condition guarantees that the plies denote the same vertices. If c coincided with no x_j, one would reach $c \in supp(E^0) = \emptyset$ (the root has no ancestors): absurd. □

The lemma is, for derivation chains, the counterpart of the fact that in [8] the gray vertices are exactly the ancestors of the current call. Exact (or superset) inheritance at a skip is what makes it work: with the deepest-blocker rule the SKIP step loses blockers, and Proposition 4.7 shows that then even completeness at the root fails.

Lemma 4.11 ($R \subseteq R_0$)**.** *Every vertex of R has an expansion in Π_0.*

Proof. Induction along a path of vertices of N from v_0. $v_0 \in R_0$ by (i). Let $v_i \in R_0$ with an expansion $E^i \in \Pi_0$, and let v_{i+1} be a child of v_i with $v_{i+1} \in N$. In E^i (which ends NIL, in the mode that examines all children) the child v_{i+1} is examined; its outcome is not proven, because it would be permanent while $v_{i+1} \in N$; so the mode is EXPAND or SKIP, and the used expansion is in Π_0 by (ii), or it is GRAY, and then $v_{i+1} \in supp(E^i)$ is a chain vertex (Lemma 4.10), which has an expansion in Π_0. In every case $v_{i+1} \in R_0$. □

Lemma 4.12 (Descent)**.** *For every $x \in R_0$, $eval(x) \neq g$.*

Proof. Suppose, for a contradiction, that some $x \in R_0$ has $eval(x) = g$, take one with $d_g(x)$ minimal, and let $E \in \Pi_0$ be an expansion of x (it ends NIL, so x is not a sink). If x is an OR vertex, by Definition 2.8 there is a child c with $eval(c) = g$ and $d_g(c) = d_g(x) - 1$; in E the outcome of c is not TRUE (otherwise E would end TRUE) and not FALSE (Lemma 4.2), hence NIL. If x is an AND vertex, all children have $eval = g$ and $d_g \leq d_g(x) - 1$, and at least one child c has outcome NIL in E (that is what makes E end NIL). In both cases c is a child examined NIL in $E \in \Pi_0$: in the mode EXPAND or SKIP the used expansion is in Π_0; in the mode GRAY, $c \in supp(E)$ is a chain vertex (Lemma 4.10). So $c \in R_0$ with $eval(c) = g$ and $d_g(c) < d_g(x)$, against the minimality. □

Proof of Theorem 4.8. $R \subseteq R_0$ (Lemma 4.11) and every vertex of R_0 has $eval \neq g$ (Lemma 4.12); the equivalence with the model checker was argued above. □

The proof uses only three properties of the dejavu: a skip produces NIL, the parent inherits a superset of the blockers of the skipped entry, and the inherited blockers denote vertices on the current path. It therefore holds for variant B as well. Empirically, every lemma was checked by a model checker on about $5 \cdot 10^7$ random searches before the proof was written, and the "natural" statements that random testing refutes (for instance, that *every* expansion ending NIL has support within its ancestors of the final NIL path) are recorded in the repository as what cannot be the way.

Remark 4.13 (The published Java). The procedure of Section 3.3 keeps, for each vertex, the set of blocking *vertices* instead of plies and a prefix hash, and skips a white child when all its blockers are on the path. The three properties above hold for it directly: a skip yields NIL (line 32 reads $w.win = \text{NIL}$), the virtual hit of lines 25–30 gives the parent $w.dejavu \setminus \{v\}$ and every ancestor down to each blocker the same vertex (which is the eager form of (R4)), and the skip condition identifies the blockers as vertices, so the case (V3) of Lemma 4.3 and Lemma 4.10 need no prefix. Lemmas 4.2–4.12 and the two theorems therefore apply to it; the only difference is that it skips more often (the same blockers in a different order are enough), which loses no completeness at the root. Table 3 is the empirical counterpart.

5 Traversal: from depth-first to df-pn

5.1 A generic core with WinTree's semantics

The first C++ core is a template on the game (G provides the successors, the turn, the terminal classification and a 64-bit key of the current position) implementing the procedure of Section 3.3 with the four-valued outcome of Section 2, budgets (expansions, moves, depth, deadline, a cooperative stop flag), a proof table and the dejavu of Section 4. The table is a hash table of fixed size with buckets of four entries of two 64-bit words, $w_0 = key \oplus data$ and $w_1 = data$, both atomic with relaxed loads and stores: a reader accepts an entry only if $w_0 \oplus w_1$ equals its key, so that a torn write or another position fails the check without locks (the technique of Stockfish's transposition table, with a full 64-bit key). A proven entry carries the goal, the depth d of the proof tree (an upper bound on d_g, Theorem 3.6), the validity interval on the fifty-move counter of Section 6.3 and a witness move; an UNKNOWN_CYCLE entry carries (p, hash); an UNKNOWN_BUDGET entry carries the residual depth tried. The invariants of the core are those of [8] made explicit: a vertex becomes proven only from proven children or terminals, never from a skip, a budget cut or a heuristic value; a budget cut produces UNKNOWN_BUDGET, which propagates as NIL but raises a sticky flag on every ancestor, so that no UNKNOWN_CYCLE entry ever hides a cut; unresolved outcomes are never stored as definitive.

Differential tests run the core against the oracle and the transcription on the 10^4 graphs of Section 3.4 (same generator, same seeds, verified identical): 0 proven entries in contrast with the oracle, 0 incomplete roots, 0 failures of Theorem 4.8, 0 proof depths below d_g, and 15 858 roots not TRUE, the same number as the transcription; a second search on the full table repeats every outcome with 0 expansions; with random budgets (1–50 nodes, depth 1–15) every interrupted search ends with an UNKNOWN_BUDGET root with the cut flag and 0 wrong proven entries.

5.2 Why the traversal changed

The first quantitative gate asked whether the depth-first core, used as a mate solver, can certify at least 90 % of the mates that Stockfish finds in the same time, with a number of nodes at most three times that of a textbook df-pn [7] written as a minimal control (threshold search with $1 + \varepsilon$, its own table, no graph-history treatment). The suite is `mates2000` [11], 2000 positions with a known mate; the protocol gives each solver 10 s of CPU on one thread, a 256 MiB table zeroed before each position, no tablebases, and "warm" ordering of the children, that is, the order produced by a short Stockfish search of the position (an in-process instance, read-only). Stockfish's `go mate` with the same 10 s solves 1789 of the 2000 positions in the first run.

solver (warm ordering)	certified / solved by Stockfish	median nodes	p90
WinTree depth-first	1147/1789 = 64.1 %	7.07×	332×
textbook df-pn	1583/1789 = 88.5 %	1×	1×
df-pn with WinTree semantics	1719/1788 = 96.1 %	0.21×	0.57×

Table 5: Gate G1 on `mates2000`, 10 s per position, one thread. "Solved by Stockfish" is `go mate` within 10 s (1789 positions in the first run, 1788 in the second). Nodes are `do_move` calls, relative to the textbook df-pn of the same run. With cold ordering the three solvers certify 59.4 %, 87.9 % and 96.0 %; the third one without the pn/dn initialization heuristic 92.9 %.

Table 5 gives the outcome. The depth-first core fails the gate: 64.1 % certified, with a median of 7.07× the nodes of the control and a tail of 332× at the 90th percentile. The reason is structural: a depth-first search of an AND/OR graph commits to the first child and backtracks only on failure, while a best-first solver spends its nodes where the proof is cheapest. Two ablations on 200 positions confirm what matters: stopping an AND vertex at its first NIL child in the mode that looks for wins ("a child not yet refuted denies the win of this node in this iteration") is indispensable (121 solved with it, 28 without), the dejavu is irrelevant on mates, and the order of the children matters (random 49, cold 120, warm 127 of 200).

The decision at the gate was to adopt df-pn as the traversal of the production core and to keep WinTree as semantics: the four-valued outcome; the rule that a repetition is never a stored proof or refutation (a child that repeats a position

of the path gets proof number ∞ and disproof number 0 for the *selection*, but the refutation that follows from it is *tainted*: an OR vertex is tainted if one of its children's refutations is, an AND vertex chooses a clean refutation if it has one and is tainted otherwise; tainted refutations stay in the table for the search but are UNKNOWN_CYCLE entries, never PROVEN_NOWIN, while proofs never use repetitions and are clean by construction); the validity intervals of Section 6.3; the depth cap (245 plies in the win mode, as mate solvers do, a tainted refutation beyond it); and the verifier. What does not survive is the depth-first recursion itself and the dejavu, which has no effect on mates. Proof and disproof numbers are initialized from the information Stockfish has: at an AND vertex (defender to move) the proof number is the number of legal replies, at an OR vertex the disproof number is the number of legal moves (the rule of [1]), checks, captures with non-negative static exchange and the transposition-table move get a lower initial proof number, and the warm order breaks ties. With this core the gate is passed: 96.1 % of what Stockfish solves is certified, with 0.21× the nodes of the control in the median and 0.57× at the 90th percentile, and 202 positions that Stockfish does not solve in 10 s are solved with a certificate. The 69 misses are 64 budget cuts (df-pn spent its $8.5 \cdot 10^5$ expansions without closing) and 5 tainted refutations at the root, the expected loss of completeness of reused tainted refutations, a graph-history effect [3, 2, 6] that the core accepts on purpose: it loses candidates, never produces a false certificate. By mate length, the misses are 0 of 237 at $|N| \leq 5$, 14 of 821 at 6–7, 21 of 465 at 8–10, 34 of 265 beyond 10; the depth-first core missed more than half beyond $|N| = 8$.

On the synthetic graphs the df-pn core has 4140 PROVEN_WIN roots against 4142 for the depth-first core, 0 proven entries against the oracle and 0 failures of Theorem 4.8; the two wins missed are tainted refutations reused along a path where the repetition does not occur. A later addition re-searches a found proof with a decreasing depth cap (first $d - 2$, then bisection) within the remaining budget: on the mate suite the number of positions whose certified depth equals the mate length declared by the suite rises from 704 to 1567 of 1934, all certified.

6 Chess: what makes a certificate valid

The abstract model has no fifty-move counter, no repetition rule and no tablebases. This section states what the implementation does to make a PROVEN_WIN a forced win *in the regulation game*, and a PROVEN_NOWIN "the opponent cannot force a win" under the same rules. The adapter uses the position, the legal-move generator and the Syzygy probing code [4] of Stockfish without copies.

6.1 State key and repetitions

The identity of a vertex is the position as the rules define it (pieces, side to move, castling rights, a legal en passant capture): this is exactly Stockfish's Zobrist key before the fifty-move counter is mixed in, so the proof table uses

that key, and the same position with different counters shares its proof when the intervals below allow it. Inside a search a repetition of a position of the current path is a gray vertex: not a draw, but "not usable in a proof". For a win this is exact (the attacker does not win by repeating); for a non-win it is prudent (the defender need not claim a draw, he only needs not to lose). A position that already occurred twice in the game before the root is a draw the defender can claim, and is treated as a draw sink. A proof computed with one history stays valid as long as the game follows the proof tree; the verifier below replays the tree with the actual history before a move is played by proof, and after an irreversible move no earlier position can repeat.

6.2 Terminals and tablebases

A position with no legal move is a mate (the side not to move wins) or a stalemate (draw); at a counter above 99 plies the fifty-move draw applies, with a mate on the hundredth ply taking precedence, as in Stockfish; a triple repetition with the game history is a draw; dead positions are draw sinks in the cases that can be decided (king against king or king and knight, kings and bishops all on the same color). Chess960 castling is supported. Tablebases are sinks under the conditions Stockfish uses for probing (counter zero, no castling rights, piece count within the tables): a WDL of $+2$ for g is a PROVEN_WIN with tablebase leaves, a WDL of at most $+1$ for g ("win, but draw under the fifty-move rule") is a PROVEN_NOWIN when the fifty-move rule is in force; a proof whose leaves are tablebase wins is reported as a win in the tablebase band of scores, never as a mate.

6.3 The fifty-move rule as a constraint on every proof

A proof tree with sixty reversible moves is not a regulation win, and a refutation that ends in a fifty-move draw is valid only if the counter was high enough. Every proven entry therefore carries a validity interval on the counter. For PROVEN_WIN, $rev(v)$ is the maximum number of consecutive reversible plies from v along the proof tree before an irreversible move or a terminal (a sink gives 0, a reversible move to w gives $1 + rev(w)$, an irreversible move gives 1, an AND vertex takes the maximum over its children); the entry is valid for a position with counter c if $c + rev(v) \leq 100$. For PROVEN_NOWIN that uses fifty-move draw sinks, $req(v)$ is the minimum counter for which the refutations stay valid ($100 - k$ for a sink reached after k reversible plies, the maximum over the paths used); the entry is valid if $c \geq req(v)$. Tablebase sinks are reached only with counter zero and are compatible by construction. An entry outside its interval is not used and the vertex is searched again (the new result has a wider interval and overwrites the old one). Two bytes per entry and one addition per vertex suffice; for example, king and queen against king with counter 98 gives a PROVEN_NOWIN with $req = 98$ (no mate within two plies), and with counter 0 a PROVEN_WIN with $d = 15$.

6.4 Proof objects and the verifier

The search produces candidates; certainty comes from a verifier that checks an explicit proof object on reconstructed positions. This is the idea of Lemma 4.5 of [8] (a winning tree is verifiable by induction) extended to draws.

1. *Winning tree* for PROVEN_WIN: extracted from the table from the root (at the vertices of g the stored move, at the opponent's vertices all legal moves) and replayed path by path with Stockfish's `do_move`; at every vertex the terminal classification uses the *actual* counter and history of the path, so the fifty-move rule and the triple repetition are checked exactly without *rev*; at the vertices of g the move is legal and leads to a vertex of the tree, at the opponent's vertices every legal move does; the leaves are mates given by g or tablebase wins reached by an irreversible move without castling rights. The cost is the size of the tree times the branching factor, with no dependence on hash keys: the table only serves to find the chosen move.

2. *Draw-strategy graph* for PROVEN_NOWIN: a closed set S of positions with, for every vertex of S where the opponent of g moves, a chosen move. The closure check: at every vertex of S where g moves, all legal moves lead into S or to terminals not won by g; at every vertex where the opponent moves, the chosen move is legal and leads into S or to such a terminal; no terminal of S is a win of g. If the game stays in S forever it ends by repetition or by the fifty- or seventy-five-move rules: a draw. The certificate is therefore independent of counter and history; the only sinks admitted in S are mates given by the opponent of g, stalemates, dead positions and tablebase positions not won by g (a WDL of at most $+1$, valid at every counter). An UNKNOWN_CYCLE root of a complete search is the candidate from which S is extracted, as Theorem 4.8 says.

Two levels of trust follow. Entries used as bounds inside alpha-beta are *search-grade*: no replay, and a hash collision or a proof made stale by the history is worth a collision of the transposition table today. Statements printed as "proven" and moves chosen by proof are *certificate-grade* and pass through the verifier. Every certificate emitted in the experiments below went through it, and a failure of the verifier is a bug of the prover by definition; none occurred.

6.5 Correctness on positions

The chess gate G0 runs the prover on 10^4 random positions with 3 to 5 pieces *without tablebases*, compares every proven outcome with the Syzygy WDL value as an oracle, and verifies every certificate. With the df-pn core and 10^6 `do_move` per search: 4138 positions resolved (41.4 %), 1772 PROVEN_WIN (1770 winning trees certified, 2 entries replaced in the table before extraction), 2366 PROVEN_NOWIN (all draw strategies certified), 2916 UNKNOWN_CYCLE roots of which 2309 with a closed and verified draw strategy, 2946 budget cuts, and 0 disagreements with the oracle; in total 6445 certified outcomes (64.5 %) against

3696 (37.0 %) for the depth-first core at comparable work. The unresolved positions are endgames with mobile pieces (queen against queen, rook against bishop) where a proof without tablebases needs the whole graph of positions.

7 Integration into Stockfish 19

The integration keeps Stockfish's search untouched and adds a proof table consulted by it, and a prover run at the root. With the prover off, the search is identical: the standard benchmark gives the same node count (the signature 2 497 913 of Stockfish 19). The hooks are few.

- *Options.* `Prover` (`off`, the default; `auto`; `mate`; `draw`), `ProverHash` (MiB of the proof table, 16 by default), `ProverBudget` (per mille of the time the time manager allots to the move, 100 by default, 30 in the later experiments), `ProverMaxNodes` (a cap in `do_move` calls, 10^6), `ProverProbeDepth` (6), `ProverAdvantage` (centipawns, 0) and a log switch.

- *Root.* Before the helper threads start, on the main thread, the prover runs synchronously in the direction "win for the side to move" with a budget equal to the per-mille share of the allotted time, if that share is at least 10 ms, and the node cap. In `auto` it starts only on a trigger: a `go mate` command, a previous score that is already decisive (a mate score or the tablebase band), or, if `ProverAdvantage` is positive, a previous score of at least that many centipawns. A root that ended without outcome is remembered by key and not retried with the same budget. `mate` and `draw` force the prover at every move for analysis.

- *Probe.* In the main search, at every node of depth at least `ProverProbeDepth` that is not the root and not an excluded-move search, the table is probed and a valid proven entry is turned into a value and a bound exactly as a Syzygy result is: a PROVEN_WIN for the side to move is a lower bound at a mate-in-$(ply + d)$ score (or in the tablebase band if the leaves are tablebase wins), for the other side an upper bound at the mated score; a PROVEN_NOWIN is a draw score with the bound on the right side (the side that cannot win fails low at the draw score, so a certified fortress changes the move played, not only the printed text). The probe is skipped while the table has no proven entry, which is what makes it free in practice (Section 8.2).

- *Choice and output.* A PROVEN_WIN for the side to move, certified by the verifier, moves its move to the front of the root moves, sets the score to mate-in-d (or to the tablebase band), marks the result exact, and prints the principal path of the proof tree; the search is stopped after the next completed iteration, as `go mate` does. Only certificate-grade outcomes are printed (`proven mate in N`, `proven win (tablebase leaves)`, `proven draw-or-better`, and their counterparts), never a mate for a proof with tablebase leaves. The prover's `do_move` calls are added to the node count.

What was not done, deliberately: no prover thread (the second phase of the design, conditional on a strength gain that did not come), no second search in the opposite direction in the same move, no certification pass at the end of the search, no adaptive budget. Under a clock only a PROVEN_WIN for the side to move is certified and printed, because it is the only outcome that changes the move; the others remain search-grade bounds through the probe.

8 Experiments

8.1 Protocol

All games are played between the hybrid binary and itself with `Prover=off`, so that the only difference is the prover; the engines use one thread, a 16 MiB transposition table and 5-man Syzygy tablebases, with `fastchess` [5] as the arbiter, four games in parallel on an Intel Core i7-4960HQ (four cores, 16 GB). Openings come from a book of 20 084 positions generated by random legal moves and filtered by evaluation, or from the endgame sets described below; every pair of games is played with reversed colors. Strength tests are sequential probability ratio tests [12] with the pentanomial model on pairs and the bounds $[-3, 1]$ Elo ("no regression"); samples of fixed size report the Elo estimate with its 95 % interval and the pentanomial counts $[LL, LD{+}DL, DD{+}LW{+}WL, WD{+}DW, WW]$ for the prover's side. The conversion experiments report the difference of points scored by the side in advantage between `auto` and `off`, per pair, with a bootstrap interval over positions.

8.2 Cost of the probe (G4)

The probe is the only hook that runs inside the search. On the standard benchmark, with the table empty and the trigger off, it costs -0.24 % of nodes per second with `ProverProbeDepth` $= 6$ (-1.3 % with 4, -2.5 % with 1; ten alternated runs pinned to a core). In games at 60+0.6 it cost more: $-1.27\,\% \pm 0.82\,\%$ of nodes per second over 400 games with the probe always active and the trees identical (same depths, nodes and moves in 121 control searches: it is only speed). Skipping the probe while the table has no proven entry, and allocating the table at the first start of the prover rather than at option time, removed it: $+0.51\,\% \pm 0.78\,\%$ over 400 games (median -0.14 %), with the intervals of the two measurements disjoint. With proven entries in the table the probe is active again and costs as in the first measurement; on fixed positions at depth 30 it is -0.3 to $-0.4\,\% \pm 0.6\,\%$ for probe depths 6, 8 and 10.

8.3 Fortresses (G2)

The hypothesis to test was that the prover certifies draws beyond the coverage of the tablebases. The set is built automatically: 168 positions with six pieces where Stockfish with the prover off evaluates the position beyond ± 150

centipawns while the 6-man tablebases say draw; the prover may use only sinks with at most five pieces, so it must prove, not look up. With 10 s per position, 59 of 168 (35.1 %) are certified, the same at 100 s, and *all of them by reduction to tablebase leaves* (17 with a two-node graph); without tablebases, 0 of 168. A second set of 23 fortresses from the literature with 7 to 10 pieces gives 1 of 23 with 5-man tables, 2 of 23 with 6-man tables even at 100 s, and 0 without tables; the unresolved ones explore 4 to 5 million expansions (about 140 million `do_move`, 85–93 s of CPU) without closing the graph. A local re-search inside the verifier of the draw graph raises the first count from 59 to 63. The hypothesis is not supported: the certified draws in practice are reductions to regions covered by the tablebases. A collateral observation is that Stockfish 19 at 1 s already evaluates 20 of the 23 literature fortresses within ±120 centipawns (median 38), so the engine's weakness on fortresses is smaller than the literature suggests.

8.4 Playing strength (G3)

Fast games. The first test, at 10+0.1 with the default options (budget 100 per mille, node cap 10^6, probe depth 6), accepted the hypothesis of no gain: -2.15 ± 2.43 Elo over 13 574 games (likelihood of superiority 4.2 %); a control with the probe only gave -0.58 ± 5.09 over 3000 games, and a control of the binary against itself $+3.47 \pm 5.32$. The prover produced a certified outcome in 10 games of 13 574. The analysis that followed changed the reading: at 10+0.1 the median time allotted to a move is 63 ms, the prover's share (6–10 ms at 100 per mille, 2–3 ms at 30) is below the 10 ms start threshold, and the log confirms 0 starts of the root prover in 20 400 games; the time losses were stalls of the environment, symmetric between the two engines (24 against 22, $p = 0.88$). The regression measured in the first test cannot come from the prover and is noise of the bench. A second test at 10+0.1 with the restricted trigger (only `go mate` or a decisive score), the budget at 30 per mille and the memory of failed roots reached its cap of 20 000 games undecided at -1.39 ± 2.02 Elo, compatible with zero and with the cost of the probe.

Long games from the opening. A sample of 1500 games at 60+0.6 from the opening book gave -5.79 ± 5.53 Elo (pentanomial $[0, 86, 604, 59, 1]$, likelihood of superiority 2.0 %): the prover ran (a share of 11–55 ms per move) but rarely, 16 starts in 10 games with 2 certified wins; the prover's side searched 1.2 % fewer nodes per move. With ten interventions in 1500 games the effect of the prover is not measurable this way.

Endgame positions chosen by evaluation. A set of 481 positions (135 with 6–7 pieces, 246 with 8–10 pieces and at most three pawns, 100 long mates) chosen for being won but difficult, played in 1000 pairs at 60+0.6 with adjudication only by tablebases: $+0.17 \pm 1.13$ Elo, pentanomial $[1, 1, 995, 2, 1]$. The prover started in 1870 of the 2000 games and certified 5086 outcomes in 953 games, at 138 ms per game without outcome (about 0.3 % of the thinking time), with no loss on

time due to it. The five unequal pairs are two stalls of the environment and three long mates (two for the prover's side with a certified mate, one against). The prover works and certifies, and Stockfish converts anyway.

Positions that Stockfish does not convert. The previous result suggested choosing positions by *outcome* rather than by evaluation: positions where the engine with the prover off fails to convert an advantage. Candidates were the first position at a score of at least +3.00, or a mate score, for a side that then did not win, taken from the games already played (205 positions; none from the 2300 long games from the opening book, where the evaluation never reaches +3 in a draw) and from 6000 new games at 2+0.02 (262), plus 66 long mates of the suite that the engine does not see in 1.5 s. Each candidate was then played twice between two copies of the engine with the prover off at 60+0.6 (1054 games), keeping the positions that the side in advantage failed to win at least once; 37 positions lost twice by the side "in advantage" were removed, as the +3 of the fast game was simply wrong. The set has 133 positions: 19 endgames with 6–10 pieces, 101 positions with 11–32 pieces and 13 long mates; 120 of them were drawn twice in the filter.

Five hundred pairs at 60+0.6 gave $+0.69 \pm 3.85$ Elo, pentanomial $[0, 15, 468, 17, 0]$, 56 wins, 54 losses and 890 draws for the prover's side. The conversion difference is +0.002 points per pair, with bootstrap interval $[-0.010; +0.014]$; the side in advantage won 40 games with each engine. The prover started 771 times in 111 games (73 pairs of 500), certified 298 outcomes in 56 games (165 mates, 133 wins with tablebase leaves), and spent 12.2 s in total on the 473 starts without outcome (110 ms per game with a start, about 0.2 % of the thinking time). The reading is structural: the prover starts only after a decisive score, every game that reaches such a score is won by both engines (56 of 56 with the prover, 54 of 54 without), and where the engine does not convert, the search never reaches that score and the prover never starts (427 pairs of 500 without a single start). In the 32 unequal pairs the prover's log always shows a line, but as an effect of the trigger, not as a cause: in the 15 pairs won by the prover's side with a certificate the previous move was already at a mate score or at +199, and the balance is compatible with the noise of the filter (11 unequal pairs of 133 between two copies of the engine with the prover off).

An earlier trigger. The only way to give the prover room where the engine does not convert is a trigger that fires before the decisive score. The option `ProverAdvantage` $= N$ starts the prover at every move whose previous score is at least N centipawns for the side to move; the budget and the memory of failed roots are unchanged, so above the threshold the prover restarts at every move. A pre-analysis of the previous sample shows the limit: in the 920 draws the maximum evaluation of the side in advantage has median 0.00 (+0.82 at the 90th percentile), at +3.00 the trigger would have fired in 7 % of the games with room in 1 %. The sample was therefore targeted: the 23 positions of the previous set in which the side in advantage touched +1.00 without reaching a decisive

score (2 endgames, 20 positions with 11–32 pieces, 1 mate), with $N = 100$, 500 pairs at 60+0.6, the same protocol. Result: -6.60 ± 7.28 Elo, pentanomial $[0, 67, 385, 48, 0]$; conversion -0.019 points per pair, interval $[-0.044; +0.006]$; wins of the side in advantage 155 against 173 (the same 23 positions in the previous sample, 85 pairs with the decisive trigger: -0.006, $[-0.066; +0.057]$). The prover started 8536 times in 511 games (8.5 per game, against 1.7 before on the same positions), in 352 of the 500 games of the prover's side in advantage, in the median at the second ply. Of the 6923 starts due to the threshold, 6916 ended at the budget and 7 certified (with a previous evaluation between +8.67 and +14.39, not the +1 to +3 of the threshold); the other 562 certified outcomes came, as before, with the score already decisive. The cost was 801 ms per game with a start (about 0.9 % of the thinking time); nodes per second $+0.37\,\% \pm 0.51\,\%$. Every game with a decisive score was won (155 of 155 with the prover, 174 of 174 without). Of the 115 unequal pairs, 48 were for the prover's side (46 with a certificate, 3 of them before the decisive score) and 67 against (none with a certificate); in the 107 pairs converted by both engines the prover certified in 104: the proof follows the win, it does not produce it. With 11 to 32 pieces and an evaluation between +1 and +10, the proof tree does not fit a budget of 50–140 ms (about 35 000 `do_move`). A signal of +0.060 seen after 50 pairs did not survive.

9 Conclusions

What is established by execution or proof.

1. The reference implementation of [8] has three reproducible defects: P1 (the two-loop order is unsound), P2 (the dejavu is incomplete at the root) and P2b (an unsound FALSE at the root that follows from P2). The procedure of Section 3 there is correct in every test. The corrected procedure of Section 3.3 is published and shows no violation on 10^4 graphs.

2. The dejavu with exact inheritance of the blockers is complete at the root (Theorem 4.4) and satisfies Theorem 4.8 (Theorem 4.8); the rule that inherits only the deepest blocker is not complete (Proposition 4.7), and random testing does not see it.

3. WinTree's semantics (four-valued outcomes, repetitions as unusable rather than as draws, validity intervals for the fifty-move rule, an independent verifier) is implementable in C++ over an unmodified Stockfish and produced no false certificate in any test. As a traversal, the depth-first recursion is inferior to a textbook df-pn on mates; df-pn with WinTree's semantics and Stockfish's ordering information certifies 96 % of what `go mate` solves, with verified proofs, and solves 202 positions of 2000 that Stockfish does not solve in 10 s.

4. The draws that can be certified in practice are reductions to regions covered by the tablebases, not fortresses beyond them.

5. Inside Stockfish the system costs nothing measurable as a probe and gains nothing measurable as strength, at 10+0.1 and at 60+0.6, on openings, on endgames chosen by evaluation and on positions chosen by outcome. The mechanism is now understood: the prover starts after a decisive score, which the engine converts alone; an earlier trigger starts it where it should, and the proof tree does not fit a per-move budget.

What is not established: an engine stronger than Stockfish at any time control measured; an advantage in fortresses beyond the tablebases; any effect of the prover in play distinguishable from noise. The result of the project is a *certification tool*: mates and reductions to tablebases with verifiable proof objects, built over Stockfish 19 intact, with the prover off by default for play and used as an analysis mode (`Prover=mate`, `Prover=draw`, or the standalone solver). Opening room for the prover in play would need a budget larger by orders of magnitude than a move's share of the clock, or a target other than the win of the whole position (a subgoal that a proof can close within the budget); both are outside what was measured here. On the theoretical side, the df-pn core reuses tainted refutations without the prefix condition of the dejavu, and a version of Theorem 4.8 for it, with an admissible inheritance, is open; so is the measurement, rather than the estimate, of the loss due to prefix-hash collisions.

Acknowledgments

The implementation, the experiments and the drafting of this article were carried out with the assistance of Claude (Anthropic). Every number reported here was recomputed from the files of the repository [9], where each result is reproducible with one command.

A Reproducibility

The repository [9] contains the unmodified sources of Stockfish 19 [10], the published Java (`tests/wintree-java`, with the harness of Section 3.4: `run.sh`), the generic cores and their tests (`src/wintree`: `make test`), the chess solver with the verifier, the G0 and G1 benchmarks (`src/wintree-chess`), the hybrid engine (`src/stockfish-hybrid`, a modified copy whose differences from the unmodified copy are the hooks of Section 7), the fortress corpus and scripts (`tests/g2`), the game scripts and books (`tests/g3`, `tests/books`), the raw results (`tests/results`: CSV files, logs, compressed PGN files, per-position provenance of the outcome set), the proof document of Section 4 with the full trace of the counterexample, and the design and report document that records every decision with its date. Software: g++ 16 (C++17), OpenJDK 25, fastchess alpha 1.8.2, Syzygy tables up to six pieces; the Stockfish network file is not distributed and is downloaded as Stockfish itself does. Hardware: Intel Core i7-4960HQ at 2.60 GHz, four cores (eight threads), 16 GB of memory. The

whole repository is released under the GNU General Public License, version 3 or later, the license of Stockfish.

References

[1] L. Victor Allis, Maarten van der Meulen, and H. Jaap van den Herik. Proof-number search. *Artificial Intelligence*, 66(1):91–124, 1994.

[2] Dennis M. Breuker, H. Jaap van den Herik, Jos W. H. M. Uiterwijk, and L. Victor Allis. A solution to the GHI problem for best-first search. *Theoretical Computer Science*, 252(1–2):121–149, 2001.

[3] Murray Campbell. The graph-history interaction: on ignoring position history. In *Proceedings of the 1985 ACM Annual Conference on the Range of Computing*, pages 278–280, 1985.

[4] Ronald de Man. Syzygy endgame tablebases. `https://github.com/syzygy1/tb`.

[5] Disservin and contributors. fastchess: a command-line tool for managing chess games with engines. `https://github.com/Disservin/fastchess`. Version alpha 1.8.2.

[6] Akihiro Kishimoto and Martin Müller. A general solution to the graph history interaction problem. In *Proceedings of the Nineteenth National Conference on Artificial Intelligence (AAAI-04)*, pages 644–649, 2004.

[7] Ayumu Nagai. *Df-pn Algorithm for Searching AND/OR Trees and Its Applications*. PhD thesis, University of Tokyo, 2002.

[8] Fabio Romano. How to program a never-losing chess engine. arXiv:2607.09715v2 [cs.GT], 2026. `https://arxiv.org/abs/2607.09715`.

[9] Fabio Romano. WinTreeChess: source code, test harnesses and results. `https://github.com/fabioromano1/WinTreeChess`, 2026. GNU General Public License, version 3 or later.

[10] The Stockfish developers. Stockfish 19, source code. `https://stockfishchess.org`. GNU General Public License, version 3 or later.

[11] Joost VandeVondele. matetrack: track the mate finding effectiveness of Stockfish. `https://github.com/vondele/matetrack`. The suite `mates2000.epd`.

[12] Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.